Interpolation in Mathilda vs. Mathematica

Accuracy, performance, and feature comparison of `Interpolation` / `InterpolatingFunction`

2026-06-05

1. Introduction

Mathilda is a pico computer-algebra system written in C99 that recreates the core architecture and evaluation semantics of the Wolfram Language. This document compares Mathilda's `Interpolation` builtin — together with the `InterpolatingFunction` object it produces — against the reference implementation in Mathematica.

The comparison covers three axes:

1. **Accuracy** — does Mathilda produce the *same interpolant* as Mathematica, and how closely does the interpolant track the underlying function?
2. **Performance** — how do construction and evaluation times scale?
3. **Comprehensiveness** — which forms, methods, and options does each system support?

All Mathematica figures were produced with `wolframscript` (Mathematica 14.x) and all Mathilda figures with the `feature/sum-stages-0-3` build of `./Mathilda`, on the same macOS machine. Every numeric value quoted below was generated by running the *identical* input through both systems.

1.1 What Mathilda implements

`Interpolation[data]` builds an `InterpolatingFunction` object that can be applied like a function. Mathilda supports:

- **Default piecewise-polynomial interpolation** via sliding-window Newton divided differences (`InterpolationOrder -> 3` by default).
- `InterpolationOrder -> n` for any order (linear through high-degree).
- `Method -> "Spline"` (natural / cyclic cubic spline) and `Method -> "Hermite"` (tensor-product Hermite basis).
- **Supplied derivatives**: 1-D data `{{x}, f, f', f'', ...}` to any order, and n-D data with supplied gradient and Hessian (mixed-partial tensors).
- `PeriodicInterpolation -> True` for all methods, including a cyclic cubic spline, in any dimension.
- **Vector- and array-valued samples** `{x, {v1, v2, ...}}`, combinable with supplied derivatives.
- **n-dimensional regular grids.**

- **Arbitrary precision** through MPFR, alongside machine precision.
- **Derivatives of the interpolant** (`f'`), extrapolation with the `InterpolatingFunction::dmval` warning.

2. Accuracy

2.1 The interpolants agree with Mathematica

The headline result is that Mathilda's default method reproduces Mathematica's default `InterpolatingFunction` **digit for digit**. Both systems use sliding-window divided differences of order 3.

Sampling the Runge function $f(x) = 1/(1 + 25x^2)$ at 21 equally-spaced nodes on $[-1, 1]$ (step 0.1) and evaluating at off-node points:

x	Mathilda	Mathematica	$\lvert\Delta\rvert$
0.05	0.93125	0.93125	0
0.25	0.391827	0.3918269230769231	$< 10^{-7}$
0.45	0.164605	0.1646054376657825	$< 10^{-7}$
0.65	0.0864057	0.08640566994527152	$< 10^{-7}$
0.85	0.052438	0.052437996243490145	$< 10^{-7}$
0.95	0.0424715	0.0424715060189533	$< 10^{-7}$

Mathilda and Mathematica are not merely *close* — they compute the same polynomial on each cell. The only discrepancies are below Mathilda's six-digit display precision.

2.2 Tracking the underlying function

Because the two systems produce the same interpolant, they share the same approximation error relative to the true function. For the Runge data above:

x	interpolant	exact $f(x)$	error
0.05	0.93125	0.941176	9.9×10^{-3}
0.25	0.391827	0.390244	1.6×10^{-3}
0.45	0.164605	0.164948	3.4×10^{-4}
0.65	0.0864057	0.0864865	8.1×10^{-5}
0.85	0.052438	0.052459	2.1×10^{-5}
0.95	0.0424715	0.0424403	3.1×10^{-5}

The larger error near $x = 0$ is the well-known Runge phenomenon, identical in both systems.

2.3 Spline method

`Method -> "Spline"` on the same Runge data:

x	Mathilda	Mathematica
0.05	0.938866	0.9388662126816522
0.45	0.164865	0.16486466302504507
0.95	0.0425342	0.042457716912143846

Mathilda's natural cubic spline agrees with Mathematica's spline to full display precision in the interior. The small divergence at $x = 0.95$ (last interval) reflects a different *end condition*: Mathilda uses natural (zero second derivative) boundaries, while Mathematica's B-spline uses not-a-knot. This is a boundary-only effect; interior cells coincide.

2.4 Hermite method

`Method -> "Hermite"` with supplied first derivatives, data `{{{0.},0.,1.},{{1.},1.,1.},{{2.},8.,12.}}`:

x	Mathilda	Mathematica
0.5	0.5	0.5
1.5	3.125	3.125

Exact agreement — the Hermite basis is uniquely determined by the supplied values and slopes.

2.5 Periodic interpolation

Data sampled over one full period of $\sin(2\pi x/8)$ at integer nodes $0, \ldots, 8$, with `PeriodicInterpolation -> True`:

evaluation	Mathilda	Mathematica
$f(2.5)$	0.916053	0.9160533905932737
$f(10.5)$	0.916053	0.9160533905932737

Both systems wrap identically ($f(2.5) = f(10.5)$, period 8) and agree to full machine precision.

2.6 Vector-valued, n-D, and derivatives

Test	Input	Mathilda	Mathematica
Vector-valued	`{{0,{1,2}},{1,{3,4}},{2,{7.5,7.8125}},{3,{13,14}}}` at 1.5	`{7.5, 7.8125}`	`{7.5, 7.8125}`
2-D grid	$\sin(x+y)$ on $\{0..3\}^2$, at $(1.5, 1.5)$	0.1351	0.13510002236965674
Derivative f'	sin data, $f'(1.5)$	0.0704247	0.07042474693418055

Vector-valued samples match exactly; the 2-D and derivative results agree to Mathilda's display precision.

2.7 Arbitrary precision

Mathilda routes through MPFR when the data or the evaluation point carries extended precision. Interpolating 30-digit sin data and evaluating at a 30-digit argument:

```
Mathilda :  0.975987231040146037270997180316 72
Mathematica: 0.975987231040146037270997180316893
```

The two agree to ~28 significant figures; the trailing digits differ only by the last-place rounding of the 30-digit inputs. `Precision[...]` reports `30.103` in Mathilda, matching Mathematica's `WorkingPrecision` accounting.

3. Performance

Timings below are wall-clock seconds from each system's `Timing[]`, on identical data (sin sampled at integer nodes). "Build" is one `Interpolation[...]` call; "eval/pt" is the per-point cost of evaluating the resulting object.

N nodes	Mathilda build	MMA build	Mathilda eval/pt	MMA eval/pt	eval slowdown
100	0.16 ms	0.16 ms	89 µs	2.2 µs	~40×
1000	1.96 ms	1.0 ms	1.11 ms	2.0 µs	~560×
2000	2.56 ms	2.0 ms	2.81 ms	2.0 µs	~1400×

Construction is competitive — Mathilda is within ~2× of Mathematica across all sizes.

Evaluation is where Mathilda is markedly slower, and the slowdown *grows with* N. Mathematica's per-point cost is flat at ~2 µs (it locates the bracketing cell with an $O(\log N)$ binary search). Mathilda's per-point cost scales **linearly with** N (89 µs → 1.11 ms → 2.81 ms as N goes 100 → 1000 → 2000), because each evaluation performs an $O(N)$ linear scan to find the bracketing window and re-runs the full evaluator fixed-point loop. For small and moderate node counts this is imperceptible; for large grids with many evaluations it is the dominant cost.

This is the clearest optimization opportunity in Mathilda's implementation: replacing the linear window search with a binary search (and caching the last cell) would bring per-point evaluation to near-constant time.

4. Comprehensiveness

4.1 Feature matrix

Capability	Mathilda	Mathematica
Default piecewise-polynomial (order 3)	Yes	Yes
`InterpolationOrder -> n`	Yes	Yes
`Method -> "Spline"`	Yes (natural/cyclic cubic)	Yes (B-spline)
`Method -> "Hermite"`	Yes	Yes
Supplied 1-D derivatives, any order	Yes	Yes
Supplied gradient / Hessian (n-D)	Yes	Yes

Capability	Mathilda	Mathematica
`PeriodicInterpolation -> True`	Yes	Yes
Vector- / array-valued samples	Yes	Yes
n-dimensional regular grids	Yes	Yes
Arbitrary precision	Yes (MPFR)	Yes
Derivative of interpolant (`f'`)	Yes	Yes
Extrapolation + `dmval` warning	Yes	Yes
Nested grid-array input form	No (needs `Flatten`)	Yes
Property queries `f["Domain"]`, `f["Grid"]`	No	Yes
`ListInterpolation`	No	Yes
`Integrate` / `NIntegrate` of interpolant	No	Yes
Complex-valued samples	No	Yes
Unstructured / scattered data	No	Yes (limited)

4.2 Where the systems match

Mathilda covers the entire *core* surface of `Interpolation`: all methods, all data forms (value-only, derivative-supplied, scalar, vector, array), every dimension, both precisions, periodicity, and differentiation of the result. On every supported form the numeric output matches Mathematica, usually to display precision and frequently bit-for-bit.

4.3 Where Mathematica goes further

- **Input convenience.** Mathematica accepts the nested grid array `Table[{{x,y}, f}, {x,...}, {y,...}]` directly; Mathilda requires a flattened list of `{coord, value}` pairs (`Flatten[..., 1]`). It also offers `ListInterpolation` for data given as a bare value array on an implied grid.
- **Object introspection.** Mathematica's `InterpolatingFunction` answers property queries — `f["Domain"]`, `f["Grid"]`, `f["Methods"]`, `f["ValuesOnGrid"]` — which Mathilda does not implement (the object exposes only its abbreviated `{domain, <>}` print form).
- **Downstream symbolic/numeric integration.** Mathematica can `Integrate` and `NIntegrate` an `InterpolatingFunction`; Mathilda's `Integrate` does not yet recognize the object.
- **Complex-valued data.** Mathematica interpolates complex samples (`fc[1.5] = 3.25 + 1.0625 I`); Mathilda currently leaves complex-valued `Interpolation` unevaluated.
- **Scattered / unstructured data.** Mathematica handles certain non-grid data sets; Mathilda requires a regular product grid.

5. Summary

On its supported surface, **Mathilda's `Interpolation` is a faithful recreation of Mathematica's**:

- **Accuracy** is excellent. The default method is digit-for-digit identical to Mathematica; Spline, Hermite, periodic, vector-valued, n-D, derivative, and arbitrary-precision results all match to

display precision (boundary-condition choices aside).

- **Construction speed** is competitive (within ~2×).
- **Evaluation speed** is Mathilda's weak point: an $O(N)$ per-point window scan makes it 40×–1400× slower than Mathematica's $O(\log N)$ lookup on large grids. This is a localized, well-understood optimization target.
- **Comprehensiveness** spans the full core feature set. The remaining gaps are convenience and ecosystem features — nested-grid input, object property queries, integration of interpolants, complex-valued data, and scattered data — rather than gaps in the interpolation mathematics itself.

For a pico CAS, the fidelity of the interpolation engine to Mathematica's — across methods, dimensions, derivatives, and precision — is the notable result.